Device Configurator Framework - Instructor Lab Guide

Objectives

Part 1: Verify the Scripting Environment
Part 2: Build the Configurator Application
Part 3: Use the Configurator Application
Part 4: Improvements & Reflections

Estimated completion: 75 minutes

Background / Scenario

In this lab, students will build a menu-driven Python application that interacts with a Cisco CSR1000v device. The primary objective of this lab is to demonstrate simple code modularity. Not all possible features or functions are implemented. Instead, students are encouraged to think about ways to expand and improve upon the given application.

Important Note: Code in this document may be presented for visual clarity. Student's code should follow proper indentation - refer to Python Style Guidelines (PEP8).

Success Indicators

Upon completion of the lab, students will have assembled a functional Python application that connects to a Cisco router to execute simple "show" commands. Upon providing a username and password, the application connects to the router, outputs resulting text from the "show" command(s), and returns to the previous menu without errors.

<u>Part 1: Verify the Scripting Environment.</u>

Step 1a.
Open the DEVASC VM console to verify the Python environment and install prerequisites if missing.

In the DEVASC VM console, open the Terminal and type:

```
python3 -version
```

Ensure Version 3 is present. Verify netmiko and requests packages are installed:

```
pip show netmiko
pip show requests
```

If they are missing, install them with `pip install netmiko` and `pip install requests`.

Step 1b.
Identify and record the CSR1000v IP address.

Open the console of the CSR1000v virtual machine. Type the command

```
show ip interface brief
```

```
CSR1kv>show ip int br
Interface              IP-Address      OK? Method Status
GigabitEthernet1       192.168.197.131 YES DHCP   up
```

The figure above is an example only. Record the IP address shown for `GigabitEthernet1`. Close the CSR1000v console.

<u>Part 2. Build the Configurator Application.</u>

For reference, you will use Visual Studio Code to create the following files in the working directory:

```
capstone_main.py
capstone_dev_mgt.py
capstone_inv_mgt.py
get_current_device.py
current_device.json
device_list.json (optional)
/cisco_cmd
     Interfaces_rest.py (recommended)
     show_ip_int_br.py
     show_ver_uptime.py (optional)
```

Students facing time constraints may skip implementation of interfaces_rest.py and show_ver_uptime.py. However, students are still expected to answer questions pertaining to these functions.

Step 2a.
Create the project working directories.

In the DEVASC VM terminal, type:

```
mkdir /home/devasc/labs/capstone-project
mkdir /home/devasc/labs/capstone-project/cisco_cmd
```

Step 2b.
Create the main executable, `capstone_main.py`.

In the DEVASC VM terminal, open **Visual Studio Code** and copy or type the following into a blank tab:

```
from capstone_inv_mgt import inv_main
from capstone_dev_mgt import dev_main

def main():
    while True:

        try:

            print("*** MAIN MENU ***" + "\n" "________" + "\n")

            print("1. Inventory Management" + "\n" + "2. Device
Management" + "\n" + "9. Quit" + "\n" + "________\n")

            user_input = input("Selection: ")
            user_input = int(user_input)

            if user_input == 1:
               inv_main()
            elif user_input == 2:
               dev_main()
            elif user_input == 9:
                print("Goodbye.")
                break
            else:
                print("Invalid number. Try again.")

        except:
            print("Error: Invalid Entry.")

if __name__ == "__main__":
    main()
```

Click **File**, **Save As**. Navigate to the directory

```
/home/devasc/labs/capstone-project
```

Name the file `capstone_main.py`. Click **Save**. Do not close Visual Studio Code. Open a new tab for the next step.

Step 2c.
Create the `capstone_dev_mgt.py` subscript.

Using the same techniques described in **Step 2b**, create `capstone_dev_mgt.py` with the code below:

```
from get_current_device import get_current_device
from cisco_cmd.show_ip_int_br import show_ip_int_br
from cisco_cmd.show_ver_uptime import show_ver_uptime
from cisco_cmd.interfaces_rest import interfaces_rest

def dev_main():

    while True:

        try:

            print("*** DEVICE MANAGEMENT MODE ***" + "\n" "________" +
"\n")

            var_router = get_current_device()

            print("Current Device: "+ var_router["host"])

            print("1. show ip int br" + "\n" + "2. show ver uptime" +
"\n" + "3. interfaces-state (REST)" + "\n" + "9. Return to Main" + "\n" +
"________\n")

            user_input = input("Selection: ")
            user_input = int(user_input)

            if user_input == 1:
               show_ip_int_br()
            elif user_input == 2:
               show_ver_uptime()
            elif user_input == 3:
               interfaces_rest()
            elif user_input == 9:
                print("Going back to Main.")
```

```
                break
            else:
                print("Invalid entry! Try again.")

        except:
            print("Unexpected error in: dev_main")
            break
```

Question 2c: What is the purpose of the line `user_input = int(user_input)`? What would happen without it?

Type your answer here: **This variable assignment converts a text entry into an integer. Without it, the user's input would never match the if/elif conditions, breaking functionality.**

Step 2d.
Create the `capstone_inv_mgt.py` subscript.

Using the same techniques described in **Step 2b**, create `capstone_inv_mgt.py` with the code below:

```
def inv_main():
    while True:

        try:
            print("*** INVENTORY MANAGEMENT MODE ***" + "\n" "________" +
"\n")
            input("Nothing to see here. Hit enter to return to the Main
Menu.")
            break

        except:
            print("Unexpected error in: inv_main")
```

Step 2e.
Create the `get_current_device.py` subscript.

Using the same techniques described in **Step 2b**, create `get_current_device.py` with the code below:

```
def get_current_device():
    import json

    with open('current_device.json', 'r', encoding='utf-8') as f:
        current_device = json.load(f)
```

```
    return(current_device)
```

Step 2f.
Create the `current_device.json` file. This single dictionary entry contains the IP address of the CSR1000v router.

Using the same techniques described in **Step 2b**, create `current_device.json` with the snippet below, using the IP address of your router recorded in **Step 1b**.

```
{
    "device_type": "cisco_ios",
    "host": "192.168.197.131",
    "port": "22"
}
```

Question 2f: What other device information might be useful to contain in this file?

Type your answer here: **Open ended, but students could note that hostname, make, and model would be beneficial to track.**

Step 2g.
Create the `cisco_cmd\show_ip_int_br.py` script.

Using the same techniques described in **Step 2b**, create `cisco_cmd\show_ip_int_br.py` with the code below:

```
from netmiko import ConnectHandler
from get_current_device import get_current_device
import getpass

var_router = get_current_device()

def show_ip_int_br():

   while True:

    try:
            usernm = input("Username: ")
            passwd = getpass.getpass("Password: ")
            var_router["password"] = passwd
            var_router["username"] = usernm

            with ConnectHandler(**var_router) as conn:

                print("Connecting to: " + var_router["host"])
```

```
                net_connect = ConnectHandler(**var_router)
                output = conn.send_command('show ip interface brief')
                print(output + ".\n")
                net_connect.disconnect()
                break

        except:
                print("Unexpected Error in: show_ip_int_br")
                break
```

Be sure to save the file in the cisco_cmd directory.

Step 2h (optional).
Refer to **Step 2g**. Create the `cisco_cmd\show_ver_uptime_br.py` script.

Copy and paste but modify the 22nd line from **Step 2g:**

```
output = conn.send_command('show ip interface brief')
```

to:

```
output = conn.send_command('show ver | inc uptime')
```

Keep everything else the same and save the file as `cisco_cmd\show_ver_uptime_br.py.`

Step 2i. (optional; Question 2i still required)
Create the `cisco_cmd\interfaces_rest.py` script.

Using the same techniques described in **Step 2b**, create `cisco_cmd\show_interfaces_rest.py` with the code below:

```
import requests
import pprint
import getpass
from get_current_device import get_current_device

var_router = get_current_device()

def interfaces_rest():

    while True:

        try:
```

```
            usernm = input("Username: ")
            passwd = getpass.getpass("Password: ")

            HOST = var_router["host"]
            URL_BASE = f"https://{HOST}/restconf/data"
            AUTH = (usernm, passwd)
            HEADERS = {
            'Content-Type': 'application/yang-data+json',
            'Accept': 'application/yang-data+json'
            }

            url = f"{URL_BASE}/ietf-interfaces:interfaces-state/"
            response = requests.get(url, auth=AUTH, headers=HEADERS,
verify=False)
            pprint.pprint(response.json())
            break

        except:
            print("Unexpected Error in: interfaces_rest")
            break
```

Question 2i: What is the purpose of `verify=False?` Try changing it to `verify=True` and explain the result (wait until completing **Step 3d**):

Type your answer here:**This flag tells the requests library to skip TLS certificate verification. It needs to be skipped in the lab environment which uses self-signed certificates. Changing it to True will result in an error and not being able to connect to the device.**

Step 2j.
Verify all files are present as noted at the beginning of Part 2.

In the DEVASC VM terminal, type `ls -la`. Your output should be similar to the following (absolute paths may differ):

```
devasc@labvm:~/labs/capstone-project/automation/03.28.26$ ls -la
total 36
drwxrwxr-x 4 devasc devasc 4096 Apr  4 17:10 .
drwxrwxr-x 3 devasc devasc 4096 Apr  4 14:19 ..
-rw-rw-r-- 1 devasc devasc 1271 Apr  4 14:19 capstone_dev_mgt.py
-rw-rw-r-- 1 devasc devasc  312 Apr  4 14:19 capstone_inv_mgt.py
-rw-rw-r-- 1 devasc devasc  837 Apr  4 14:19 capstone_main.py
drwxrwxr-x 3 devasc devasc 4096 Apr  4 14:19 cisco_cmd
-rw-rw-r-- 1 devasc devasc   80 Apr  4 14:19 current_device.json
-rw-rw-r-- 1 devasc devasc  190 Apr  4 14:19 get_current_device.py
drwxrwxr-x 2 devasc devasc 4096 Apr  4 14:20 __pycache__
```

```
devasc@labvm:~/labs/capstone-project/automation/03.28.26/cisco_cmd$ ls -la
total 24
drwxrwxr-x 3 devasc devasc 4096 Apr  4 14:19 .
drwxrwxr-x 4 devasc devasc 4096 Apr  4 17:10 ..
-rw-rw-r-- 1 devasc devasc  914 Apr  4 14:19 interfaces_rest.py
drwxrwxr-x 2 devasc devasc 4096 Apr  4 14:20 __pycache__
-rw-rw-r-- 1 devasc devasc  856 Apr  4 14:19 show_ip_int_br.py
-rw-rw-r-- 1 devasc devasc  861 Apr  4 14:19 show_ver_uptime.py
```

Part 3. Use the Configurator Application.

Step 3a.
Open the DEVASC VM terminal and launch the application:

```
python3 capstone_main.py
```

You should be presented with the Main Menu, indicating successful code execution:

```
devasc@labvm:~/labs/capstone-project/automation/03.28.26$ python3 capstone_main.py
*** MAIN MENU ***
--------

1. Inventory Management
2. Device Management
9. Quit
--------

Selection:
```

Insert your Main Menu screenshot here:

Step 3b.
Open Device Management mode by selecting Option #2.

```
Selection: 2
*** DEVICE MANAGEMENT MODE ***
--------

Current Device: 192.168.197.131
1. show ip int br
2. show ver uptime
3. interfaces-state (REST)
9. Return to Main
--------

Selection:
```

The output `Current Device: 192.168.197.131` indicates that the `get_current_device()` function was able to open and read from the `current_device.json` dictionary file.

Step 3c.

Execute Option #1. When prompted, enter the username of `cisco` and the password `cisco123!`

```
Selection: 1
Username: cisco
Password:
Connecting to: 192.168.197.131
Interface              IP-Address      OK? Method Status                Protocol
GigabitEthernet1       192.168.197.131 YES DHCP   up                    up      .

*** DEVICE MANAGEMENT MODE ***
--------

Current Device: 192.168.197.131
1. show ip int br
2. show ver uptime
3. interfaces-state (REST)
9. Return to Main
--------

Selection:
```

The output above shows a successful connection to the CSR1000v router using the Netmiko ConnectHandler (SSH) method.

Insert your Device Management Option #1 screenshot here:

This is the primary expected outcome of the lab.

Step 3d. (if Step 2i was completed)
Execute Option #3. When prompted, enter the username of `cisco` and the password `cisco123!`

```
Selection: 3
Username: cisco
Password:
/usr/lib/python3/dist-packages/urllib3/connectionpool.py:1004: InsecureRequestWarning: Unverified HTTPS request is being made to
 host '192.168.197.131'. Adding certificate verification is strongly advised. See: https://urllib3.readthedocs.io/en/latest/adva
nced-usage.html#ssl-warnings
  warnings.warn(
{'ietf-interfaces:interfaces-state': {'interface': [{'admin-status': 'up',
                                                    'if-index': 1,
                                                    'last-change': '2026-04-04T14:11:52.000431+00:00',
                                                    'name': 'GigabitEthernet1',
                                                    'oper-status': 'up',
                                                    'phys-address': '00:0c:29:7b:a3:2c',
                                                    'speed': '1024000000',
                                                    'statistics': {'discontinuity-time': '2026-04-04T14:10:39.000351+00:00',
                                                                   'in-broadcast-pkts': '0',
                                                                   'in-discards': 0,
```

The output above shows a successful connection to the CSR1000v router using the `requests.get()` method.

Step 3e.
Exit the application by selection Option #9 twice.

<u>Part 4. Improvements & Reflections.</u>

In this lab, students have created a small framework to build upon. Finally, we conceptually explore opportunities for improvement.

Step 4a.
At this time, Inventory Management Mode is nonfunctional and states the following:

```
*** MAIN MENU ***
--------

1. Inventory Management
2. Device Management
9. Quit
--------

Selection: 1
*** INVENTORY MANAGEMENT MODE ***
--------

Nothing to see here. Hit enter to return to the Main Menu.
```

Consider the following Inventory Management sub-functions:

1. **List Devices**. Reads the `device_list.json` file and outputs all devices.
2. **Set Current Device**. Copies a selected device to `current_device.json`.
3. **Add a Device**. Appends a device to `device_list.json.`
4. **Remove a Device**. Removes a device from `device_list.json.`
5. **Modify a Device**.

What challenges might you encounter when implementing `set_current_device()` ?

Type your answer here:**This function should copy a device that is listed in the inventory to the current_device.json dictionary entry. There are a few challenges. First, a unique identifier needs to be established for all devices in device_list. Second, users need to be prompted to input the unique identifier through the CLI. Third, a query needs to be executed to gather the selected device information. Last, the query results need to be written to the file.**

Step 4b.
Device Management Mode functions currently retrieve information from the router. How might you implement a function that sets or writes device information, such as `set_looopback0_ip()` ? (this function would prompt the user for an IP address and subnet mask to create/set the router's Loopback0 interface. Assume you're using Netmiko/SSH).

Type your answer here: **Students should provide relevant code or pseudocode related to setting IP addresses gathered from user input.**

Step 4c.
Examine the printed code that forms the Main Menu and Device Management Mode menus:

```
print("1. Inventory Management" + "\n" + "2. Device Management" + "\n" +
"9. Quit" + "\n" + "________\n")

print("1. show ip int br" + "\n" + "2. show ver uptime" + "\n" + "3.
interfaces-state (REST)" + "\n" + "9. Return to Main" + "\n" +
"________\n")
```

Each time a menu option is added, the developer needs to append options to a single line of code. What opportunities exist to streamline and modularize the menu code?

Type your answer here: **Students could suggest breaking the single line into multiple, stacked print lines so that menu options can be added vertically instead of horizontally. Alternatively, menu options could be pulled from a text file or dictionary/JSON.**

Step 4d.
Suppose you want to implement a Device Management function called `show_ip_route()`. Given the existing structure of the project, what line of code would you need to add to the very top of `capstone_dev_mgt.py`?

Type your answer here: The correct response is:

```
from cisco_cmd.show_ip_route import show_ip_route
```

End of lab.